

IMPLEMENT GRAPH TRAVERSAL ALGORITHMS

Separate implementations of Depth-First Search and Breadth-First Search using Java

Experiment 1: Depth-First Search (DFS)

Aim

To implement Depth-First Search for traversing an undirected graph from a given starting vertex.

Algorithm / Working Process

1. Create an adjacency list for the given number of vertices.
2. Read each edge and add both vertices to each other's adjacency lists because the graph is undirected.
3. Create a visited array and call DFS using the selected starting vertex.
4. Mark the current vertex as visited and display it.
5. Recursively visit every unvisited adjacent vertex.
6. Stop when no unvisited vertex is reachable from the current traversal path.

Program

```
import java.util.*;
public class GTDFS {
    static void dfs(List<Integer>[] g, int u, boolean[] v) {
        v[u] = true; System.out.print(u + " ");
        for (int w : g[u]) if (!v[w]) dfs(g, w, v);
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices: "); int n = sc.nextInt();
        List<Integer>[] g = new ArrayList[n];
        for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
        System.out.print("Enter number of edges: "); int e = sc.nextInt();
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            g[u].add(v); g[v].add(u);
        }
        System.out.print("Enter starting vertex: "); int s = sc.nextInt();
        System.out.print("DFS: ");
        dfs(g, s, new boolean[n]); System.out.println();
    }
}
```

Output

```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
3 5
4 5
Enter starting vertex: 0
DFS: 0 1 3 5 4 2
```

Result

Thus, the above program for Depth-First Search (DFS) was implemented and executed successfully, and the graph was traversed in depth-first order.

Experiment 2: Breadth-First Search (BFS)

Aim

To implement Breadth-First Search for traversing an undirected graph from a given starting vertex.

Algorithm / Working Process

1. Create an adjacency list for the given number of vertices.
2. Read each edge and add both vertices to each other's adjacency lists because the graph is undirected.
3. Create a queue and visited array, then insert the starting vertex and mark it as visited.
4. Remove the front vertex from the queue and display it.
5. For every unvisited adjacent vertex, mark it as visited and add it to the queue.
6. Repeat until the queue becomes empty, producing a level-by-level traversal.

Program

```
import java.util.*;
public class GTBFS {
    static void bfs(List<Integer>[] g, int s, boolean[] v) {
        Queue<Integer> q = new ArrayDeque<>(); q.add(s); v[s] = true;
        while (!q.isEmpty()) {
            int u = q.poll(); System.out.print(u + " ");
            for (int w : g[u]) if (!v[w]) {
                v[w] = true; q.add(w);
            }
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices: "); int n = sc.nextInt();
        List<Integer>[] g = new ArrayList[n];
        for (int i = 0; i < n; i++) g[i] = new ArrayList<>();
        System.out.print("Enter number of edges: "); int e = sc.nextInt();
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt(), v = sc.nextInt();
            g[u].add(v); g[v].add(u);
        }
        System.out.print("Enter starting vertex: "); int s = sc.nextInt();
        System.out.print("BFS: ");
        bfs(g, s, new boolean[n]); System.out.println();
    }
}
```

Output

```
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v):
0 1
0 2
1 3
1 4
2 4
3 5
4 5
Enter starting vertex: 0
BFS: 0 1 2 3 4 5
```

Result

Thus, the above program for Breadth-First Search (BFS) was implemented and executed successfully, and the graph was traversed in breadth-first order.